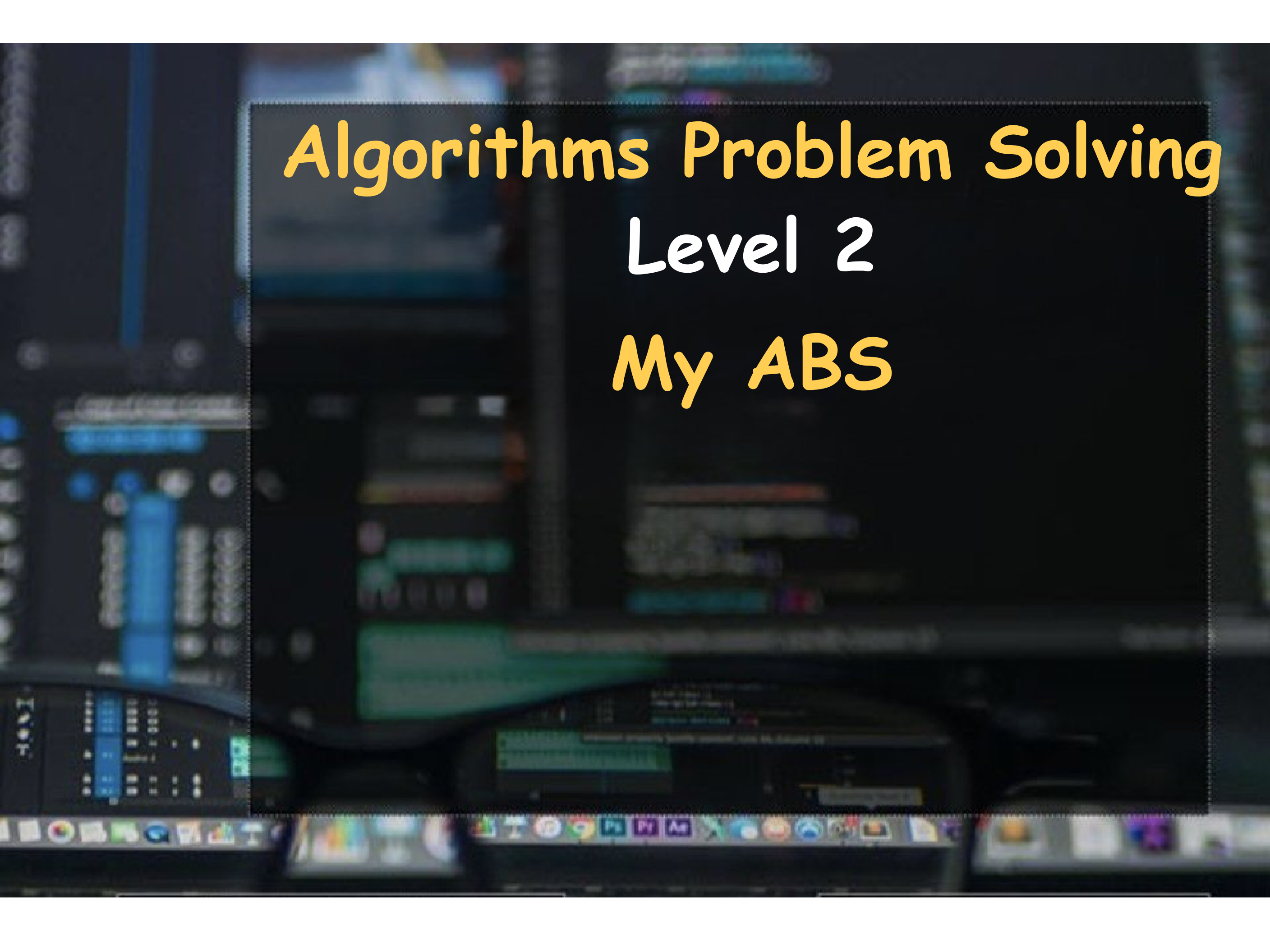




Algorithms Problem Solving

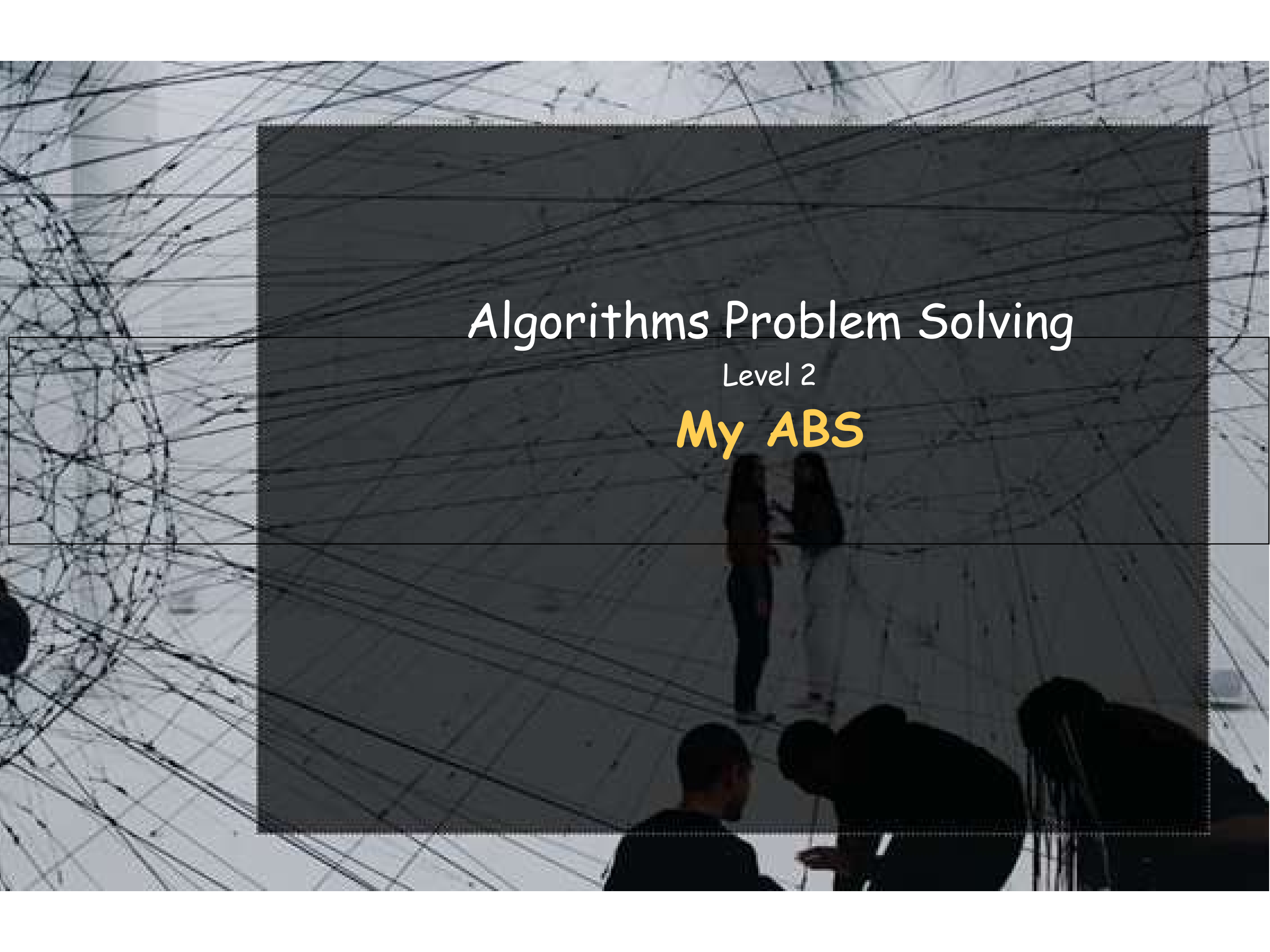
Level 2

My ABS

Algorithms Problem Solving

Level 2

My ABS



Definition

- ◆ This program calculates the absolute value of a number without using the built-in `abs()` function.
- ◆ The custom function `MyABS()` converts a negative number into its positive equivalent.
- ◆ The absolute value of a number is its distance from `0`, so it is always non-negative.

Explanation

- ◆ ``ReadNumber()`` reads a number from the user.
- ◆ ``MyABS()`` checks whether the number is positive or negative.
- ◆ If the number is greater than ``0``, it returns the number as it is.
- ◆ Otherwise, it multiplies the number by ``-1`` to make it positive.
- ◆ For example:
 `MyABS(-10)`
- ◆ Since ``-10`` is not greater than ``0``:
 $-10 * -1 = 10$
- ◆ If the input is ``10``, the function simply returns ``10``.
- ◆ If the input is ``0``, the function returns ``0``.

Steps in main()

1. Ask the user to enter a number.
2. Pass the number to `MyABS()`.
3. Check if the number is greater than `0`.
4. If it is positive, return the number.
5. Otherwise, multiply the number by `-1`.
6. Print the result from `MyABS()`.
7. Print the result from the built-in C++ `abs()` function for comparison.

💡 Example

◆ Input:

-10

◆ `MyABS(-10)` :

-10 * -1 = 10

◆ The built-in `abs()` function also returns `10`.

➡ Result

My abs Result: 10

C++ abs Result: 10

Note

- ◆ Important function:

```
int MyABS(int number)
{
    if (number > 0)
        return number;
    else
        return number * -1;
}
```

- ◆ The function does not use the built-in `abs()` function.

- ◆ The program calls `abs(number)` only at the end to compare the custom result with C++'s built-in result.

- ◆ `0` remains `0`.

⊘ Warning

- ◆ The function works correctly for normal ``int`` values.
- ◆ For the minimum possible ``int`` value, multiplying by ``-1`` can overflow because its positive equivalent cannot be represented by the same ``int`` type.
- ◆ ``cmath`` is only needed here because the program uses the built-in ``abs()`` function for comparison.



Comparison

▣ `MyABS()`

- ✓ User-defined function.
- ✓ Does not use the built-in `abs()` function internally.
- ✓ Checks the sign and converts negative values to positive.

▤ `abs()`

- ✓ Built-in C++ function.
- ✓ Directly calculates the absolute value.
- ✓ Used in this program only to compare the result with `MyABS()`.

✦ Summary

- ✓ Read a number from the user.
- ✓ Check whether the number is positive.
- ✓ Return positive numbers unchanged.
- ✓ Multiply negative numbers by `-1`.
- ✓ Return `0` as `0`.
- ✓ Compare the custom `MyABS()` result with C++'s `abs()` result.

Thank you 😊